

Lecture: Linear Search Algorithm — From Zero to Advanced

1. Introduction

Linear Search is one of the most fundamental algorithms in computer science.

It's the *starting point* for understanding how searching works before moving to more advanced techniques like Binary Search, Hashing, and Search Trees.

Definition:

A Linear Search algorithm scans each element in a collection sequentially until it finds the target or reaches the end.

Real-life analogy:

Imagine looking for your name in a list of attendance registers without it being sorted — you check one by one from the start until you find it.

2. Key Characteristics

- **Type:** Sequential Search
 - **Input Requirement:** Works on both sorted and unsorted data.
 - **Direction:** Can be from start-to-end or end-to-start.
 - **Time Complexity:** $O(n)$
 - **Space Complexity:** $O(1)$
-

3. Step-by-Step Working

1. Start from the first element.
 2. Compare it with the target value.
 3. If it matches, return the index/position.
 4. If not, move to the next element.
 5. Repeat until the element is found or list ends.
-

4. Pseudocode

LinearSearch(arr, target):

 for i = 0 to length(arr)-1:

```
    if arr[i] == target:
        return i
return -1
```

6. Complexity Analysis

- **Best Case:** $O(1)$ — if the element is found at the first position.
 - **Worst Case:** $O(n)$ — if the element is at the end or not present.
 - **Average Case:** $O(n)$ — about half the elements checked on average.
 - **Space Complexity:** $O(1)$ — only a few variables are used.
-

7. Handling Edge Cases

1. **Empty list**
 2. `data = []`
 3. `print(linear_search(data, 5))` # Output: -1
 4. **Multiple occurrences** — Decide whether to return the first or last index.
 5. **Data types** — Works with integers, strings, tuples, objects.
 6. **Case sensitivity** in strings.
 7. **Invalid inputs** — Type checking.
-

8. Variations of Linear Search

(a) Sentinel Linear Search (Optimized comparison)

- Place the target at the end temporarily to avoid checking end-of-list condition in each loop.

(b) Bidirectional Linear Search

- Search from both ends simultaneously — reduces iterations in some cases.

(c) Self-Organizing Linear Search

- Once an element is found, move it towards the front to speed up future searches (Move-to-Front or Transpose method).

10. When to Use Linear Search

☑ **Best suited for:**

- Small datasets
- Unsorted lists
- Rare searches (when performance is less critical)

✗ **Avoid if:**

- Dataset is large & frequent searches are needed — use Binary Search, Hashing, etc.
-

11. Advanced Usage

Searching in:

- **2D Lists**
 - **Linked Lists**
 - **Files & Streams**
 - **Custom Objects with Comparators**
-

12. Real-World Applications

- Finding a record in a small database.
 - Searching for a word in unsorted text.
 - Looking for a particular sensor reading in IoT devices.
 - Debugging — searching logs for a specific entry.
-

13. Summary Table

Feature	Linear Search
Works on sorted?	☑ Yes
Works on unsorted?	☑ Yes
Time Complexity	$O(n)$
Space Complexity	$O(1)$
Best Case	$O(1)$
Worst Case	$O(n)$

14. Practice Tasks

1. Modify the algorithm to return **all** occurrences of the target.
 2. Implement a **case-insensitive** string search.
 3. Write a bidirectional linear search for a list of integers.
 4. Implement a **linear search for linked lists**.
 5. Implement **self-organizing search** with Move-to-Front technique.
-

15. Closing Thought

Linear search is the “Hello World” of searching algorithms — simple yet foundational. Once mastered, it makes understanding advanced search algorithms far easier.